

	Computación	Docente: Ing. Daniel Diaz MsC.
	Programación Aplicada	Período Lectivo: 68



FORMATO DE GUÍA DE PRÁCTICA DE LABORATORIO / TALLERES / CENTROS DE SIMULACIÓN – PARA DOCENTES

CARRERA: Computación

ASIGNATURA: Programación Aplicada

NRO. PRÁCTICA:

7

TÍTULO PRÁCTICA: Sistema Bancario con Hilos (Threads)

Desarrollado por Henry Quitio

Fecha: 22-6-2026

OBJETIVO

Desarrollar un sistema bancario en Java Swing aplicando la arquitectura MVC y el uso de hilos (Threads) para el manejo concurrente de operaciones bancarias como depósitos y retiros, con el fin de reforzar los conocimientos de programación orientada a objetos y concurrencia.

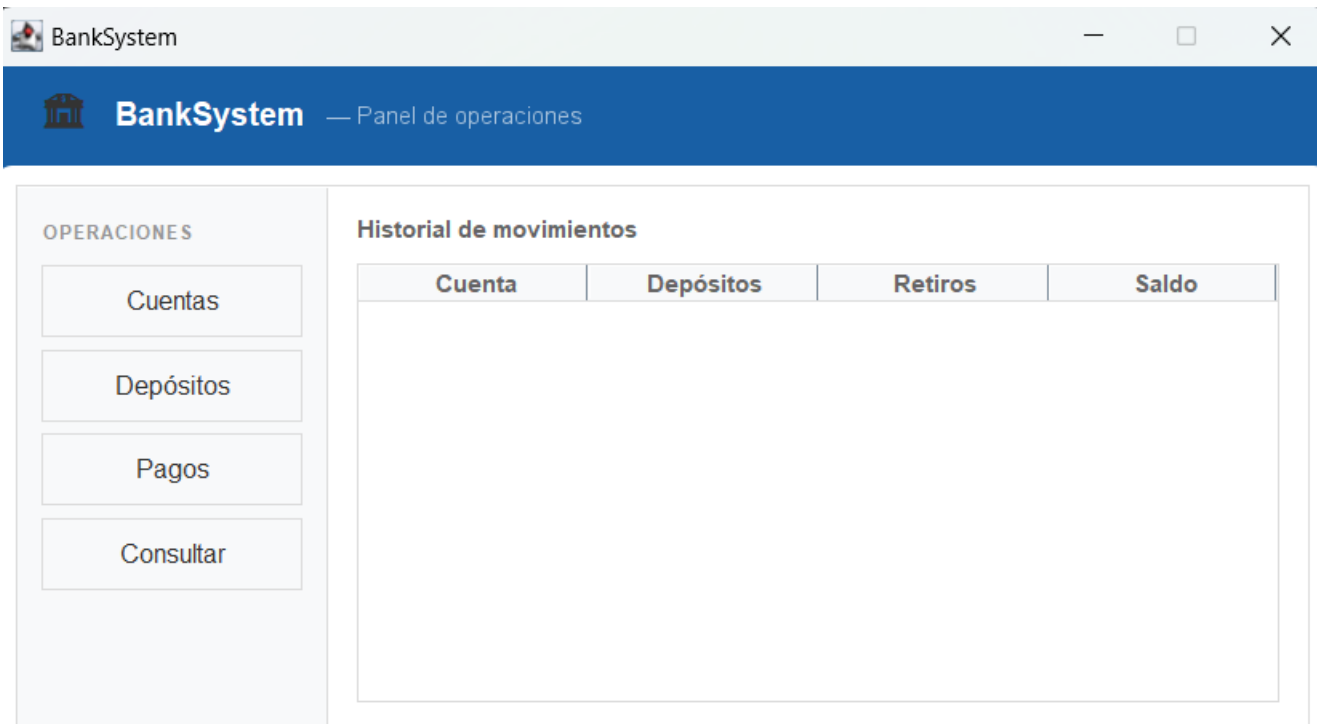
INSTRUCCIONES:

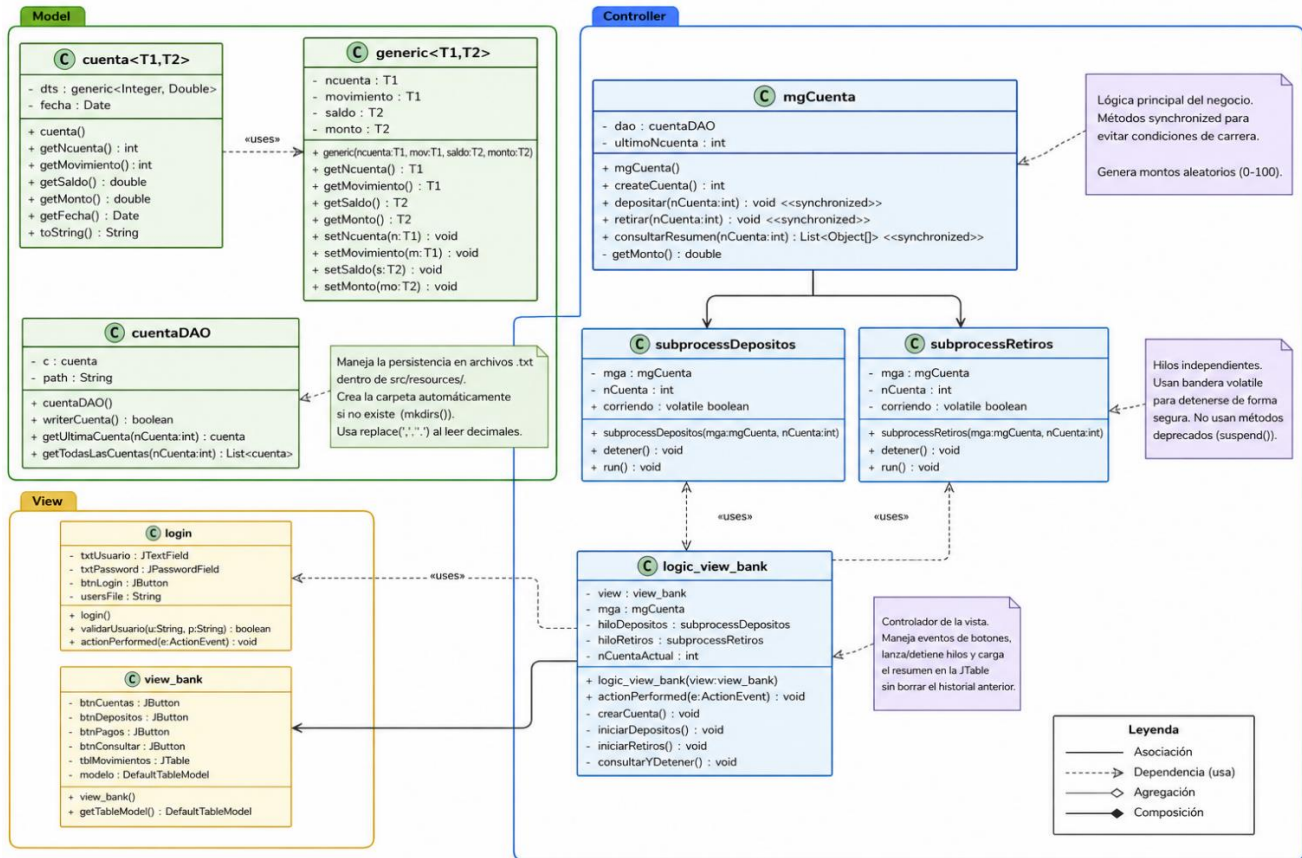
1. Aplicar los conocimientos de hilos (Thread) adquiridos en clase.
2. Construir bajo una arquitectura MVC
3. Implementar métodos sincronizados para evitar condiciones de carrera.
4. Mostrar los resultados en consola y en una JTable

ACTIVIDADES POR DESARROLLAR

Diseñar un sistema bancario en Java Swing donde el usuario puede crear una cuenta bancaria con número generado aleatoriamente, iniciar hilos independientes para depósitos y retiros automáticos, y consultar el resumen de operaciones en una tabla. El sistema debe:

- Crear cuentas con número aleatorio de 10 dígitos guardado en archivo .txt.
- Ejecutar depósitos automáticos cada 1.5 segundos en un hilo independiente.
- Ejecutar retiros automáticos cada 2 segundos en otro hilo independiente.
- Mostrar en consola cada operación con saldo anterior y nuevo saldo.
- Al presionar CONSULTAR, detener los hilos y mostrar el resumen acumulado en la JTable.





SOLUCIÓN:

Para la solución se implementaron varias clases organizadas bajo la arquitectura MVC, junto con hilos independientes para controlar depósitos y retiros de forma concurrente. A continuación, se explica cada componente.

Model/cuenta.java

Representa la entidad de una cuenta bancaria. Almacena número de cuenta, tipo de movimiento, saldo y monto mediante una clase genérica. Implementa toString() con Locale.US para garantizar punto decimal en el archivo, evitando errores de parseo en sistemas con configuración regional diferente.

```

public class cuenta {
    private generic<Integer,Double> dts;
    private Date fecha;

    public cuenta() { dts = new generic(0, 0, 0.0, 0.0); }

    public String toString() {
        return String.format(Locale.US, "%d;%d;%0.2f;%0.2f;%s%n",
            getNcuenta(), getMovimiento(), getSaldo(), getMonto(), getFecha());
    }
}

```

Model/cuentaDAO.java

Maneja la persistencia de cuentas en archivos .txt dentro de src/resources/. Crea automáticamente la carpeta si no existe mediante mkdirs(). Usa replace(',','.') al parsear para compatibilidad con configuraciones regionales que usan coma como separador decimal.

```

public class cuentaDAO {
    private final String path = System.getProperty("user.dir") + "/src/resources/";

    public cuentaDAO() {
        this.c = new cuenta();
        new java.io.File(path).mkdirs(); // crea carpeta automáticamente
    }

    public boolean writerCuenta() throws IOException {
        FileWriter out = new FileWriter(path + c.getNcuenta() + ".txt", true);
        out.write(c.toString());
    }
}

```

```

        out.close();
        return true;
    }
    public cuenta getUltimaCuenta(int nCuenta) {
        // Lee última línea del archivo
        ultima.setSaldo(Double.parseDouble(datos[2].trim().replace(",",".")));
    }
}

```

- **Controller/mgCuenta.java**

Clase de lógica principal con métodos synchronized que garantizan acceso exclusivo al archivo de cuenta entre hilos. Contiene createCuenta() que genera número aleatorio de 10 dígitos, depositar(), retirar() con validación de saldo suficiente, y consultarResumen() que acumula totales desde el archivo.

```

public class mgCuenta {
    private int ultimoNcuenta = -1;

    public synchronized void depositar(int nCuenta) {
        cuenta ultima = new cuentaDAO().getUltimaCuenta(nCuenta);
        double monto = getMonto(); // Math.random() * 100
        double nuevoSaldo = ultima.getSaldo() + monto;
        // guarda nuevo movimiento tipo 1 en archivo
        System.out.printf("[DEPÓSITO] +%.2f | Saldo anterior: %.2f | Nuevo: %.2f%n",
            monto, ultima.getSaldo(), nuevoSaldo);
    }

    public synchronized void retirar(int nCuenta) {
        if (ultima.getSaldo() >= monto) {
            // guarda movimiento tipo 2
            System.out.printf("[RETIRO] -%.2f | Nuevo saldo: %.2f%n", monto, nuevoSaldo);
        } else {
            System.out.printf("[RECHAZADO] Saldo insuficiente: %.2f%n", ultima.getSaldo());
        }
    }

    public synchronized List<Object[]> consultarResumen(int nCuenta) {
        // lee todo el archivo y acumula totalDepositos, totalRetiros, saldoActual
    }
}

```

- **Controller/subprocessDepositos.java**

Hilo independiente que ejecuta depósitos automáticos cada 1.5 segundos. Implementa la bandera volatile corriendo para detención segura del hilo sin usar métodos deprecados como suspend(). El método detener() también llama interrupt() para salir del sleep() inmediatamente.

```

public class subprocessDepositos extends Thread {
    private volatile boolean corriendo = true;

    public void detener() { corriendo = false; this.interrupt(); }

    @Override
    public void run() {
        while (corriendo) {
            try {
                mga.depositar(nCuenta);
                sleep(1500);
            } catch (InterruptedException e) { break; }
        }
    }
}

```

- **Controller/subprocessRetiros.java**

Hilo independiente que ejecuta retiros automáticos cada 2 segundos. Idéntico en estructura a subprocessDepositos, pero llama a mga.retirar(). Si el saldo es insuficiente, mgCuenta registra el rechazo en consola y no escribe en el archivo.

```

public class subprocessRetiros extends Thread {
    private volatile boolean corriendo = true;

    public void detener() { corriendo = false; this.interrupt(); }

    @Override
    public void run() {
        while (corriendo) {
            try {
                mga.retirar(nCuenta);
                sleep(2000);
            } catch (InterruptedException e) { break; }
        }
    }
}

```

- **Controller/logic_view_bank.java**

Controlador que conecta la vista con la lógica de negocio. Gestiona los ActionListeners de los cuatro botones. CUENTAS crea la cuenta en un hilo y espera con join(). DEPÓSITOS y PAGOS lanzan sus respectivos hilos como daemon. CONSULTAR detiene ambos hilos con join(3000) y carga el resumen en la JTable acumulando filas sin borrar el historial anterior.

```

public class logic_view_bank implements ActionListener {
    private subprocessDepositos hiloDepositos;
    private subprocessRetiros hiloRetiros;

    private void consultarYDetener() {
        hiloDepositos.detener();
        hiloRetiros.detener();
        hiloDepositos.join(3000);
        hiloRetiros.join(3000);
        // agrega filas sin borrar las anteriores
        List<Object[]> filas = mga.consultarResumen(nCuentaActual);
        for (Object[] fila : filas) modelo.addRow(fila);
    }
}

```

- **Funcionamiento general**

La aplicación sigue estrictamente la arquitectura Modelo-Vista-Controlador (MVC). El flujo de ejecución es:

1. El usuario inicia sesión en login.java con credenciales del archivo src/doc/user.
2. Se abre view_bank.java con los cuatro botones de operación y la JTable vacía.
3. Al presionar CUENTAS, mgCuenta genera un número aleatorio de 10 dígitos, crea el archivo .txt en src/resources/ y espera con join().
4. Al presionar DEPÓSITOS, se lanza subprocessDepositos que cada 1.5s genera un monto aleatorio (0-100) y lo suma al saldo.
5. Al presionar PAGOS, se lanza subprocessRetiros que cada 2s intenta restar un monto aleatorio, validando que haya saldo suficiente.
6. Cada operación se registra en consola con detalle de monto, saldo anterior y nuevo saldo.
7. Al presionar CONSULTAR, ambos hilos se detienen con detener()+join() y el resumen acumulado se agrega a la JTable sin borrar filas anteriores.

Resultado y Pruebas:

View_login (Vista - JFrame)

BankSystem — Login



BankSystem

Ingresa tus credenciales

Usuario

admi

Contraseña

....|

Iniciar sesión

- View_bank(Vista - JFrame)

BankSystem

**BankSystem** — Panel de operaciones

OPERACIONES

Cuentas

Depósitos

Pagos

Consultar

Historial de movimientos

Cuenta	Depósitos	Retiros	Saldo
1233385930	222,95	117,34	105,61

- Hilos (Controlador)

```

>>> Aplicación iniciada
>>> logic_view_bank iniciado
[ARCHIVO]:1233385930.txt
=====
[CUENTA CREADA] Número: 1233385930
=====
>>> Cuenta activa: 1233385930
[HILO DEPÓSITOS] Iniciado para cuenta: 1233385930
[ARCHIVO]:1233385930.txt
[DEPÓSITO] Cuenta: 1233385930 | +29,20 | Saldo anterior: 0,00 | Nuevo saldo: 29,20
[ARCHIVO]:1233385930.txt
[DEPÓSITO] Cuenta: 1233385930 | +19,62 | Saldo anterior: 29,20 | Nuevo saldo: 48,82
[HILO RETIROS] Iniciado para cuenta: 1233385930
[RECHAZADO] Cuenta: 1233385930 | Solicitado: 85,37 | Saldo insuficiente: 48,82
[ARCHIVO]:1233385930.txt
[DEPÓSITO] Cuenta: 1233385930 | +90,97 | Saldo anterior: 48,82 | Nuevo saldo: 139,79
[ARCHIVO]:1233385930.txt
[RETIRO] Cuenta: 1233385930 | -64,06 | Saldo anterior: 139,79 | Nuevo saldo: 75,73
[ARCHIVO]:1233385930.txt
[DEPÓSITO] Cuenta: 1233385930 | +36,76 | Saldo anterior: 75,73 | Nuevo saldo: 112,49
[ARCHIVO]:1233385930.txt
[RETIRO] Cuenta: 1233385930 | -53,28 | Saldo anterior: 112,49 | Nuevo saldo: 59,21
[ARCHIVO]:1233385930.txt
[DEPÓSITO] Cuenta: 1233385930 | +46,40 | Saldo anterior: 59,21 | Nuevo saldo: 105,61

```

- Final

BankSystem

BankSystem

Panel de operaciones

OPERACIONES

Cuentas

Depósitos

Pagos

Consultar

Historial de movimientos

Cuenta	Depósitos	Retiros	Saldo
1227065891	75,80	0,00	75,80
1224960877	140,00	0,00	140,00
1255664162	213,50	160,66	52,84

Requerimientos no funcionales:

Requisitos Funcionales

Gestión de cuentas y autenticación

El usuario debe iniciar sesión con credenciales almacenadas en un archivo local.

El sistema debe generar cuentas bancarias con números aleatorios de 10 dígitos.

Las cuentas creadas serán administradas por la aplicación para realizar operaciones bancarias.

Operaciones automáticas concurrentes

Los depósitos se ejecutan automáticamente cada 1.5 segundos con montos aleatorios entre 0 y 100.

Los retiros se ejecutan automáticamente cada 2 segundos con montos aleatorios entre 0 y 100.

Antes de realizar un retiro, el sistema debe verificar que exista saldo suficiente.

Consulta y almacenamiento de movimientos

Cada operación debe mostrarse en consola indicando monto, saldo anterior y saldo actual.

Todos los movimientos deben guardarse en archivos .txt dentro de src/resources/.

Al consultar, los hilos deben detenerse y la JTable debe mostrar el historial acumulado sin eliminar registros anteriores.

Requisitos No Funcionales

Arquitectura y organización

El sistema debe implementarse utilizando el patrón MVC (Modelo-Vista-Controlador).

Debe existir una clara separación entre la lógica de negocio, la interfaz gráfica y el control de eventos.

Concurrencia y seguridad

Los métodos depositar(), retirar() y consultarResumen() deben ser synchronized.

Los hilos deben controlarse mediante variables volatile, evitando métodos obsoletos como suspend().

Portabilidad y rendimiento

La aplicación debe ser compatible con diferentes configuraciones regionales usando Locale.US.

La carpeta de recursos debe crearse automáticamente si no existe.

La interfaz gráfica debe permanecer fluida y sin bloqueos durante la ejecución de los hilos.

RESULTADO(S) OBTENIDO(S):

- Se desarrolló exitosamente un sistema bancario en Java Swing con arquitectura MVC, separando correctamente la lógica (mgCuenta, cuentaDAO), la interfaz (view_bank, login) y el control de eventos (logic_view_bank).
- El uso de hilos independientes (subprocessDepositos y subprocessRetiros) permitió ejecutar operaciones bancarias concurrentes sin bloquear la interfaz gráfica.
- La sincronización con synchronized garantizó que depositar(), retirar() y consultarResumen() no generen condiciones de carrera al acceder al archivo simultáneamente desde múltiples hilos.
- El sistema de log en consola muestra en tiempo real cada operación con saldo anterior y nuevo saldo, facilitando el seguimiento de los movimientos.
- La JTable acumula el historial de consultas sin borrar registros anteriores, permitiendo comparar diferentes sesiones de operaciones bancarias.

CONCLUSIONES:

- La arquitectura MVC facilitó la organización y el mantenimiento del código. Al separar la vista, la lógica y el controlador, cualquier modificación puede realizarse de forma independiente sin afectar los demás componentes.
- El manejo de concurrencia mediante hilos demostró ser fundamental para simular operaciones bancarias en paralelo. El uso de la bandera volatile corriendo en lugar de métodos deprecados como suspend() garantiza un control seguro y moderno de los hilos.
- El uso de synchronized en los métodos de mgCuenta evitó condiciones de carrera al escribir y leer el archivo de cuenta desde múltiples hilos simultáneamente, garantizando la integridad de los datos.

RECOMENDACIONES:

- Se recomienda usar SwingUtilities.invokeLater() al actualizar la interfaz gráfica desde los hilos para evitar condiciones de carrera en el Event Dispatch Thread (EDT) de Swing.

- Se recomienda agregar validación de saldo mínimo y límites de retiro para simular restricciones bancarias reales y mejorar la robustez del sistema.
- Se recomienda implementar un sistema de niveles de concurrencia con más hilos simultáneos para evaluar el rendimiento de la sincronización bajo mayor carga.

Docente / Técnico Docente:

Firma: _____

Repositorio:

https://github.com/stivenquitio03-prog/Practica_Bank_HQ.git

